PRABHA TECHNOLOGY

prabhatechnology.com

PROJECT DOCUMENTATION

ON

GYM MANAGEMENT SYSTEM

Using Python (Flask) and React JS

In association with Prabha Technology

Submitted by

PAVAN RAMCHANDRA ZORE (Team Leader)

SUJAL SHIVAJI BHALERAO (Member)

KUNAL TULSHIDAS YANDOLE (Member)

Mentor: Rohit Sagale Sir

Internship Domain: Python Full Stack Development

Academic Year: 2026

Department: Python Full Stack Development

Table of Contents

1. Abstract

Gym management is a critical aspect of fitness centre operations, involving the tracking of members, subscription plans, active memberships, and the retail side of a gym — equipment and supplements. Traditional gym management often relies on manual registers and spreadsheets, which are error-prone, time-consuming, and difficult to scale. This project presents a Gym Management System, developed in association with Prabha Technology, using Python (Flask) for the backend and React JS for the frontend.

The system automates member registration, plan management, membership tracking, and an in-house equipment and supplement shop, all through a RESTful API backed by a relational database. It aims to reduce administrative overhead, minimise manual errors, and give gym staff a single, centralised platform for day-to-day operations. Both the Flask backend and the React JS frontend, branded as “P45 Fitness & Nutrition”, are complete and fully functional.

2. Introduction

Fitness centres and gyms need to manage a growing base of members, multiple subscription plans, ongoing memberships, and increasingly, an in-house retail counter for equipment and supplements. Manual tracking through registers or spreadsheets often results in duplicate records, missed renewals, and poor visibility into business performance.

The Gym Management System is a full-stack web application built with Python and the Flask framework for a RESTful backend API, paired with a React JS (Vite) single-page frontend that consumes that API. The system lets gym staff add, update, and manage members; define membership plans; link members to plans through trackable memberships; and manage stock for an equipment shop and a supplement shop, each with category, pricing, and stock-status tracking.

3. Objectives

- To automate member registration and record-keeping using Python and Flask.
- To design and manage subscription plans with duration and pricing details.
- To track member memberships, including start date, end date, and status.
- To manage an equipment shop and a supplement shop, including category, pricing, stock quantity, and status.
- To provide a complete RESTful API consumed by a React JS frontend.
- To build a scalable, maintainable full-stack application using modern web technologies.

4. Problem Statement

Many small and medium gyms continue to rely on manual or semi-digital methods (registers, spreadsheets) to manage members, subscriptions, and their retail counter. This leads to inaccurate records, difficulty tracking plan renewals and expiry, no centralised view of stock for equipment or supplements, and no single system tying membership data to the products a gym sells.

There is a need for a lightweight, web-based Gym Management System that centralises member data, plan details, membership records, and retail stock, while exposing this data through a clean API that powers a modern, responsive frontend. This project, developed under Prabha Technology, addresses that need using Flask for the backend and React JS for the frontend.

5. Scope of the Project

The system is designed for small to medium-sized gyms and fitness centres that need an organised way to manage members, subscriptions, and an in-house retail counter.

- Staff can add, view, update, and delete member records.
- Staff can create and manage membership plans (name, duration, price, status).
- Staff can link members to plans through memberships with start/end dates and status.
- Staff can manage an equipment shop and a supplement shop — product name, category, price, stock quantity, and status (supplements additionally track an expiry date).
- The backend exposes REST APIs consumed by the completed React JS frontend.
- Future versions can add payment tracking, attendance, and reporting dashboards.

6. Technologies Used

Technology / Tool	Purpose
Python	Primary programming language for backend development.
Flask	Lightweight backend web framework used to build REST APIs.
Flask-SQLAlchemy	Object Relational Mapper (ORM) for database models and queries.
Flask-CORS	Enables Cross-Origin Resource Sharing between the React frontend and the Flask backend.
SQLite	Relational database used to store members, plans, memberships, equipment and supplement records.
React JS (Vite)	Frontend library, built with the Vite toolchain, for a fast, component-based, single-page interface.
HTML / CSS / JavaScript	Structure, styling, and client-side logic for the frontend.
Visual Studio Code	Development and testing environment.
Postman	API testing tool used to verify backend endpoints.

7. System Architecture

The Gym Management System follows a layered, client-server architecture with a clear separation of concerns on the backend:

- Routes Layer — defines API endpoints and maps HTTP methods to controller functions (member_routes.py, plan_routes.py, membership_routes.py, equipment_routes.py, supplement_routes.py).
- Controllers Layer — contains the business logic for handling requests and responses (member_controller.py, plan_controller.py, membership_controller.py, equipment_controller.py, supplement_controller.py).
- Models Layer — defines database tables using SQLAlchemy ORM (member_model.py, plan_model.py, membership_model.py, equipment_model.py, supplement_model.py).
- Database Layer — SQLite database (gym.db) storing all persistent data.
- Frontend (React JS) — a Vite-built single-page app, branded “P45 Fitness & Nutrition”, with a tabbed interface (Members, Plans, Memberships, Equipment Shop, Supplement Shop) that consumes the REST API.

This layered structure keeps the codebase modular, making it easier to maintain, test, and extend as new features are added.

8. Methodology

- Step 1 — Requirement Analysis: identify the core entities — Members, Plans, Memberships, Equipment, and Supplements.
- Step 2 — Database Design: design tables and relationships using Flask-SQLAlchemy models.
- Step 3 — Backend Development: build REST API endpoints using Flask blueprints, controllers, and models.
- Step 4 — API Testing: verify endpoints using Postman to ensure correct request/response behaviour.
- Step 5 — Frontend Development: build the React JS (Vite) interface to consume the API.
- Step 6 — Integration: connect the React frontend to the Flask backend using Flask-CORS for cross-origin requests.

9. Database Design

9.1 Member Table

- id — Integer, Primary Key
- name — String(100), Required
- email — String(100), Unique, Required
- phone — String(15), Required
- age — Integer, Required

9.2 Plan Table

- id — Integer, Primary Key
- plan_name — String(50), Required
- duration_months — Integer, Required
- price — Float, Required
- status — String(30), Default: “Active”

9.3 Membership Table

- id — Integer, Primary Key
- member_id — Integer, Foreign Key referencing Member.id
- plan_id — Integer, Foreign Key referencing Plan.id
- start_date — String(20), Required
- end_date — String(20), Required
- status — String(30), Default: “Active”

9.4 Equipment Table

- id — Integer, Primary Key
- product_name — String(100), Required
- category — String(50), Required
- price — Float, Required
- stock_quantity — Integer, Default: 0

- status — String(30), Default: “In Stock”

9.5 Supplement Table

- id — Integer, Primary Key
- product_name — String(100), Required
- category — String(50), Required
- price — Float, Required
- stock_quantity — Integer, Default: 0
- expiry_date — String(20), Optional
- status — String(30), Default: “In Stock”

The Membership table establishes relationships with both the Member and Plan tables, allowing each membership record to be linked to a specific member and the plan they have subscribed to. The Equipment and Supplement tables are independent product catalogues that back the shop sections of the frontend.

10. Module Description

10.1 Member Module

Handles all operations related to gym members, including registration, updating member details, viewing member information, and removing members from the system.

10.2 Plan Module

Manages subscription plans offered by the gym, including plan name, duration in months, price, and active/inactive status.

10.3 Membership Module

Links members to plans, tracking the start date, end date, and current status of each membership, allowing staff to monitor active and expired subscriptions.

10.4 Equipment Module

Manages the gym’s equipment shop — product name, category (Gym Essentials, Workout Accessories, Personal Care, Useful Extras), price, stock quantity, and stock status — so staff can track what is available for sale or issue.

10.5 Supplement Module

Manages the supplement shop in the same way as the Equipment module, with the addition of an expiry date field so staff can track shelf life alongside price and stock.

11. API Endpoints

11.1 Member Endpoints

Method	Endpoint	Description
GET	/members	Retrieve all members
GET	/members/<id>	Retrieve a single member by ID
POST	/members	Add a new member

Method	Endpoint	Description
PUT	/members/<id>	Update an existing member
DELETE	/members/<id>	Delete a member

11.2 Plan Endpoints

Method	Endpoint	Description
GET	/plans	Retrieve all plans
GET	/plans/<id>	Retrieve a single plan by ID
POST	/plans	Add a new plan
PUT	/plans/<id>	Update an existing plan
DELETE	/plans/<id>	Delete a plan

11.3 Membership Endpoints

Method	Endpoint	Description
GET	/memberships	Retrieve all memberships
GET	/memberships/<id>	Retrieve a single membership by ID
POST	/memberships	Add a new membership (links a member to a plan)
PUT	/memberships/<id>	Update an existing membership
DELETE	/memberships/<id>	Delete a membership

11.4 Equipment Endpoints

Method	Endpoint	Description
GET	/equipment	Retrieve all equipment items
GET	/equipment/<id>	Retrieve a single equipment item by ID
POST	/equipment	Add a new equipment item
PUT	/equipment/<id>	Update an existing equipment item
DELETE	/equipment/<id>	Delete an equipment item

11.5 Supplement Endpoints

Method	Endpoint	Description
GET	/supplements	Retrieve all supplement items
GET	/supplements/<id>	Retrieve a single supplement item by ID
POST	/supplements	Add a new supplement item
PUT	/supplements/<id>	Update an existing supplement item
DELETE	/supplements/<id>	Delete a supplement item

12. System Features

- Add, update, view, and delete gym members.
- Create and manage membership plans with duration and pricing.
- Link members to plans and track membership status (Active/Expired).
- Manage an equipment shop and a supplement shop with category, pricing, stock, and status tracking.
- RESTful API architecture built with Flask blueprints and controllers.
- Cross-Origin Resource Sharing (CORS) enabled for frontend-backend communication.
- Complete React JS (Vite) frontend, branded “P45 Fitness & Nutrition”, with a tabbed interface across all five modules.

13. Benefits

- Reduces manual effort and paperwork in gym administration.
- Provides accurate, centralised member, membership, and retail-stock records.
- Improves efficiency in managing plans, renewals, and shop inventory.
- Cost-effective compared to commercial gym management software.
- Scalable architecture that can grow with the gym’s needs.

14. Advantages

- Clean separation of routes, controllers, and models for maintainability.
- Lightweight SQLite database, easy to set up and deploy.
- REST API can be consumed by any frontend or mobile application.
- Modular design allows easy addition of new features (e.g., payments, attendance).
- Complete, responsive React JS frontend provides a user-friendly, single-page experience across all modules.

15. Limitations

- No authentication/authorisation layer implemented yet for staff login.
- SQLite is suitable for small-scale use; larger deployments may require MySQL/PostgreSQL.
- No payment gateway integration in the current version.
- Supplier information in the Equipment module is currently stored on the client side rather than in the backend database.
- Limited to core member, plan, membership, equipment, and supplement management functionality.

16. Future Scope

- Add user authentication and role-based access control (Admin/Staff).
- Integrate online payment gateways for membership fees and shop purchases.
- Add attendance tracking using biometric or QR code check-in.
- Add automated notifications for membership renewal, expiry, and low stock.
- Move supplier data for the Equipment module into the backend database.
- Migrate to a production-grade database such as MySQL or PostgreSQL.
- Deploy the application on a cloud platform for wider accessibility.

17. Conclusion

The Gym Management System provides an efficient, structured solution for managing gym members, subscription plans, memberships, and an in-house equipment and supplement shop, using Python (Flask) for the backend and React JS for the frontend. The backend, with its clean architecture of routes, controllers, and models, is fully functional and exposes a complete REST API across five modules.

The frontend, built with React JS and Vite and branded “P45 Fitness & Nutrition”, is complete and gives gym staff an intuitive, tabbed interface to interact with the system end to end. Developed in association with Prabha Technology, the project demonstrates a full-stack implementation of a real-world business application, with clear scope for future enhancements such as authentication, payments, and analytics.

18. References

- Flask Documentation — official reference for the Flask web framework.
- Flask-SQLAlchemy Documentation — ORM reference for database models.
- React JS Documentation — official reference for building the frontend.
- Vite Documentation — official reference for the frontend build tooling.
- Python Documentation — official reference for Python programming.
- Postman Documentation — API testing reference.

Appendix A — Proposed System Flow

- 1. Start
- 2. Staff opens the P45 Fitness & Nutrition application (React JS frontend).
- 3. Staff adds/searches gym members.
- 4. Staff creates or selects a membership plan.
- 5. Staff links a member to a plan, creating a membership record.
- 6. Staff manages equipment and supplement stock through the shop modules.
- 7. System stores all data in the SQLite database via the Flask REST API.
- 8. System tracks membership status (Active/Expired) and stock status.
- 9. Staff views member, plan, membership, equipment, and supplement records.
- 10. End

Appendix B — Project Team

Student Name	Role
Pavan Ramchandra Zore	Team Leader
Sujal Shivaji Bhalerao	Team Member
Kunal Tulshidas Yandole	Team Member

Mentor: Rohit Sagale Sir

In association with: Prabha Technology (prabhatechnology.com)